VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Machhe, Belagavi – 590018, Karnataka

2022 SCHEME – BCS714D

ASSIGNMENT 2 REPORT

ON

Food and Performance Explorer: A Flask Dashboard for USDA FoodData Central Analytics

Track B – Part 2: Web Dashboard and Visualisation

Submitted as part of the coursework for Big Data Analytics

Sl. No.	Student name	USN	Marks	Signature
1	Abhishek R	1JS23CS004		
2	Adithya V	1JS23CS005		
3	Dhanush Singh G	1JS23CS048		

Group No.: 18 | Section: CSE A

Course Faculty: Sandhiya B

Department of Computer Science and Engineering

JSS ACADEMY OF TECHNICAL EDUCATION

JSS Campus Road, Dr. Vishnuvardhan Road, Bengaluru – 560060

Academic Year: 2026–2027 (ODD Semester)

Contents

Section	Page
1. Introduction and Dataset	3
2. Dashboard Architecture	4
3. Workflow Controls and Progress	5
4. MongoDB Result Presentation	6
5. Hadoop, Pig and Hive Presentation	7
6. Performance Evaluation	8
7. Conclusion and References	9

Dataset: USDA FoodData Central, October 31, 2024 CSV release. Group 18, CSE A. The report presents the implementation and recorded measurements for the linked 50,000-food development dataset.

1. Introduction and Dataset

Food & Performance Explorer is the Flask dashboard extension for the USDA analytics project. It combines result viewing, laptop workflow controls, progress diagnostics and copyable verification commands. The interface follows the three assignment themes so that raw data, transformed data and calculated answers can be explained in order.

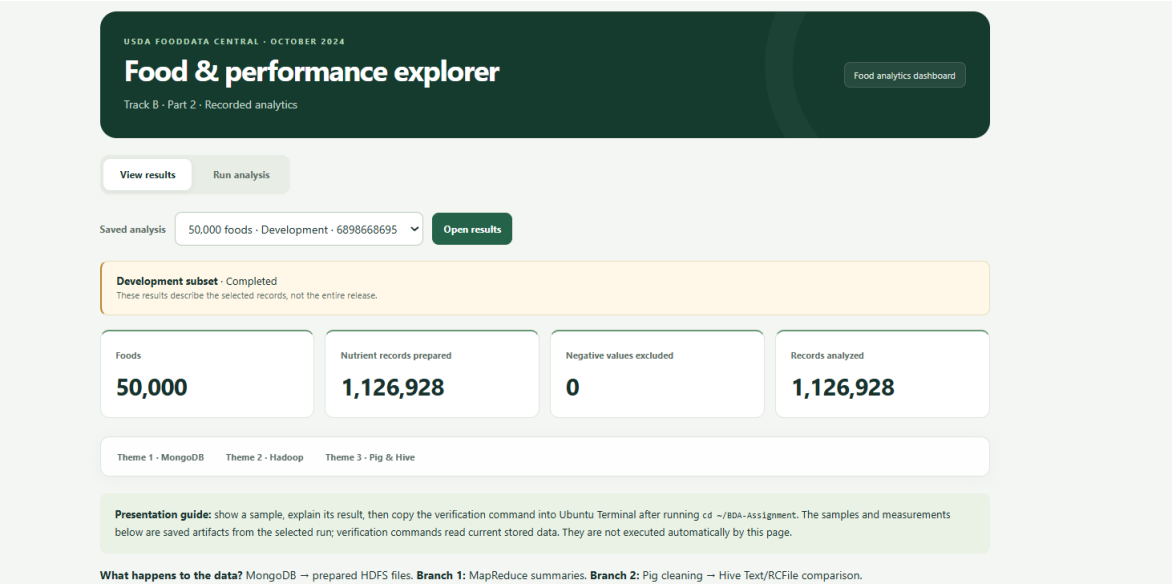


Figure 1. Completed 50,000-food run selected in the results dashboard.

The displayed experiment contains 50,000 foods and all 1,126,928 linked nutrient records, with zero negative amounts excluded. It is a labelled development subset of the October 2024 USDA release. The full source contains 2,063,876 foods and 26,848,323 food–nutrient records.

The dashboard uses Flask 3.1.2, Python, HTML templates, CSS and JavaScript. PyMongo connects the processing tools to MongoDB. Hadoop, Pig and Hive execute the analytics; the browser presents the saved outputs and controls permitted workflows. The development deployment uses one master and two worker containers on one laptop.

2. Dashboard Architecture

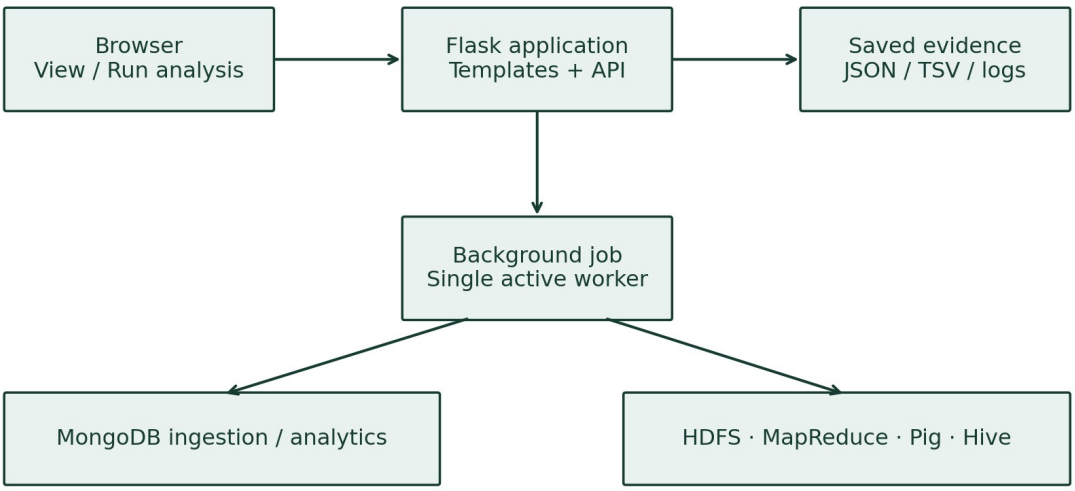


Figure 2. Browser, Flask, saved evidence and background workflow.

Flask renders the selected run and reads its JSON, TSV and log artifacts. Source fingerprints, run identifiers and counts associate the visible outputs with their inputs. Saved results remain readable while computation services are stopped; verification commands require the corresponding service to be running.

Interface	**Purpose**
GET /	Render results and analysis controls
GET /api/evidence	Read evidence for the selected run
GET /api/control/jobs	Read job states and recovery status
POST /api/control/jobs	Start a permitted predefined action
GET /api/control/jobs/<id>/log	Read job diagnostics

Only one dashboard worker is allowed at a time. Controls are restricted to predefined actions rather than arbitrary browser-supplied shell commands. The native college-cluster deployment uses view-only mode, which disables the laptop-specific Docker workflow controls.

3. Workflow Controls and Progress

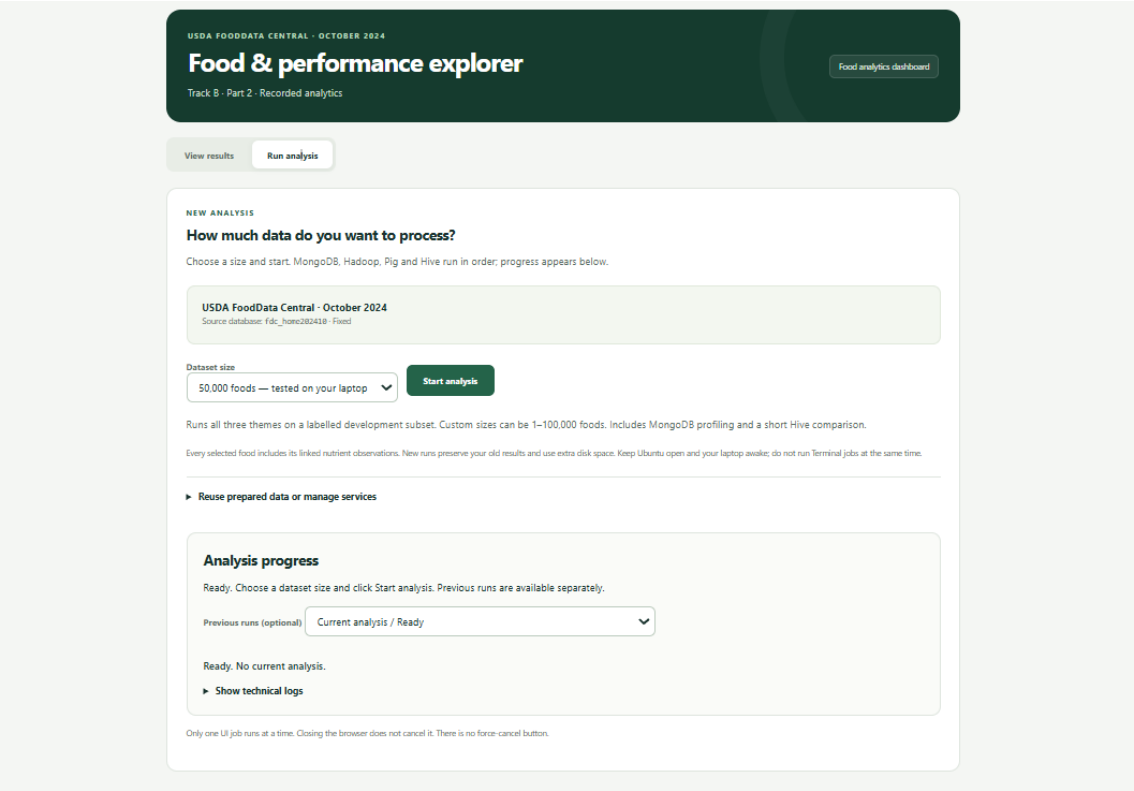


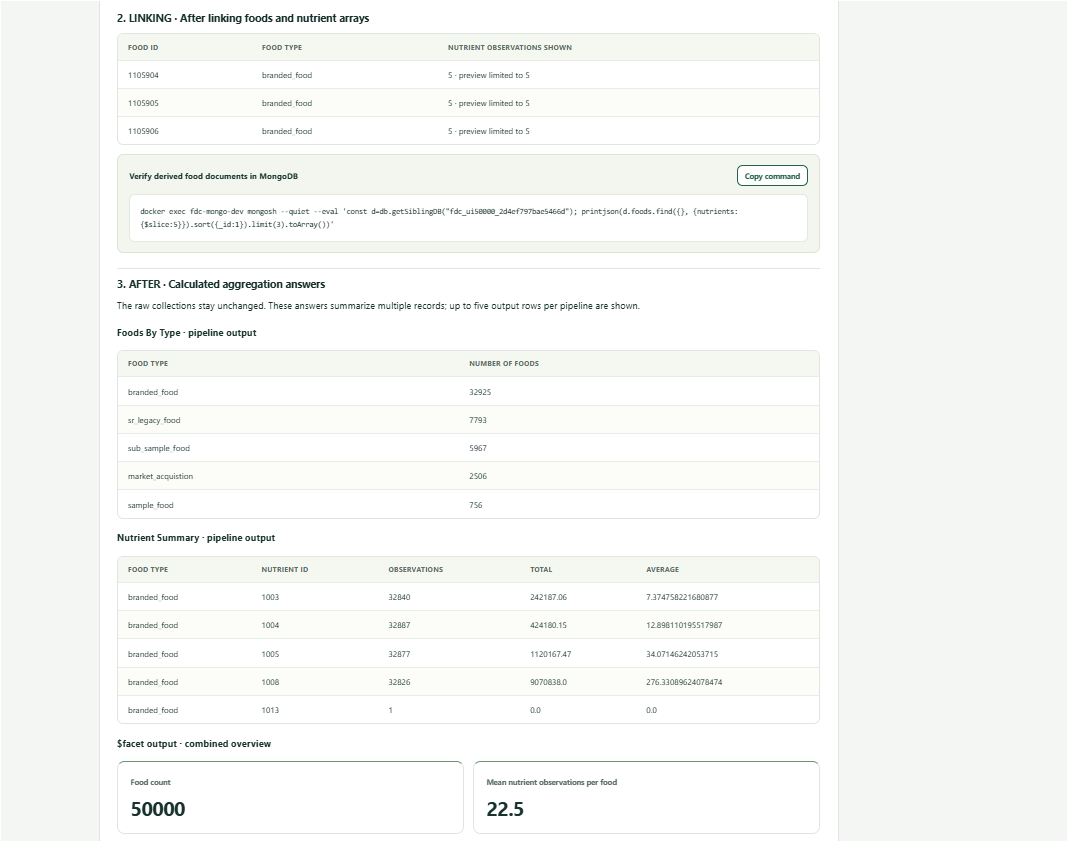
Figure 3. Run-analysis controls, fixed source database and ready progress panel.

The source database is fixed as fdc_home202410. The interface offers development sizes, custom selections from 1 to 100,000 foods, and a separate full-release route. A selected food always includes its linked nutrient observations. New run identities preserve existing saved results.

A workflow checks tools, starts MongoDB, selects records, profiles analytics, exports data, waits for Hadoop and Hive, and runs the distributed checks. Progress and technical logs update during execution. Closing the browser does not cancel a job. Interrupted jobs require the Stop services and recover action; datasets and results are retained.

4. MongoDB Result Presentation

The first theme separates original records, linked food documents and aggregation answers. Small previews keep the page readable without limiting the analysis. The displayed food count and coverage come from the faceted pipeline, while the other outputs group foods and nutrient observations.



2. LINKING · After linking foods and nutrient arrays

FOOD ID	FOOD TYPE	NUTRIENT OBSERVATIONS SHOWN
1105904	branded_food	5 - preview limited to 5
1105905	branded_food	5 - preview limited to 5
1105906	branded_food	5 - preview limited to 5

Verify derived food documents in MongoDB Copy command

```
docker exec fdc-mongo-dev mongosh --quiet --eval 'const d=db.getSiblingDB("fdc_ui50000_2d4ef797bae5466d"); printjson(d.foods.find({}, {nutrients:
{$slice:5}}).sort({_id:1}).limit(3).toArray())'
```

3. AFTER · Calculated aggregation answers

The raw collections stay unchanged. These answers summarize multiple records; up to five output rows per pipeline are shown.

Foods By Type · pipeline output

FOOD TYPE	NUMBER OF FOODS
branded_food	32925
sr_legacy_food	7793
sub_sample_food	5967
market_acquistion	2506
sample_food	756

Nutrient Summary · pipeline output

FOOD TYPE	NUTRIENT ID	OBSERVATIONS	TOTAL	AVERAGE
branded_food	1003	32840	242187.06	7.374758221680877
branded_food	1004	32887	424180.15	12.898110195517987
branded_food	1005	32877	1120167.47	34.07146242053715
branded_food	1008	32826	9070838.0	276.33089624078474
branded_food	1013	1	0.0	0.0

$facet output · combined overview

Food count
50000

Mean nutrient observations per food
22.5

Figure 4. Linked documents, grouped answers and the 50,000-food overview.

The five displayed food types include 32,925 branded_food and 7,793 sr_legacy_food records. For branded-food protein, the result contains 32,840 observations, a total recorded amount of 242,187.06 G and an average of 7.37476 G. Verification commands inspect actual MongoDB data; later preview captures are labelled separately from the original analysis.

5. Hadoop, Pig and Hive Presentation

The Hadoop section exposes the selected HDFS run directory, prepared input, verified file operations, reducer samples and recorded workers. The Pig/Hive section follows with cleaning counts, matching analytical answers and two format comparisons. The presentation distinguishes the MapReduce branch from the separate Pig-to-Hive branch.

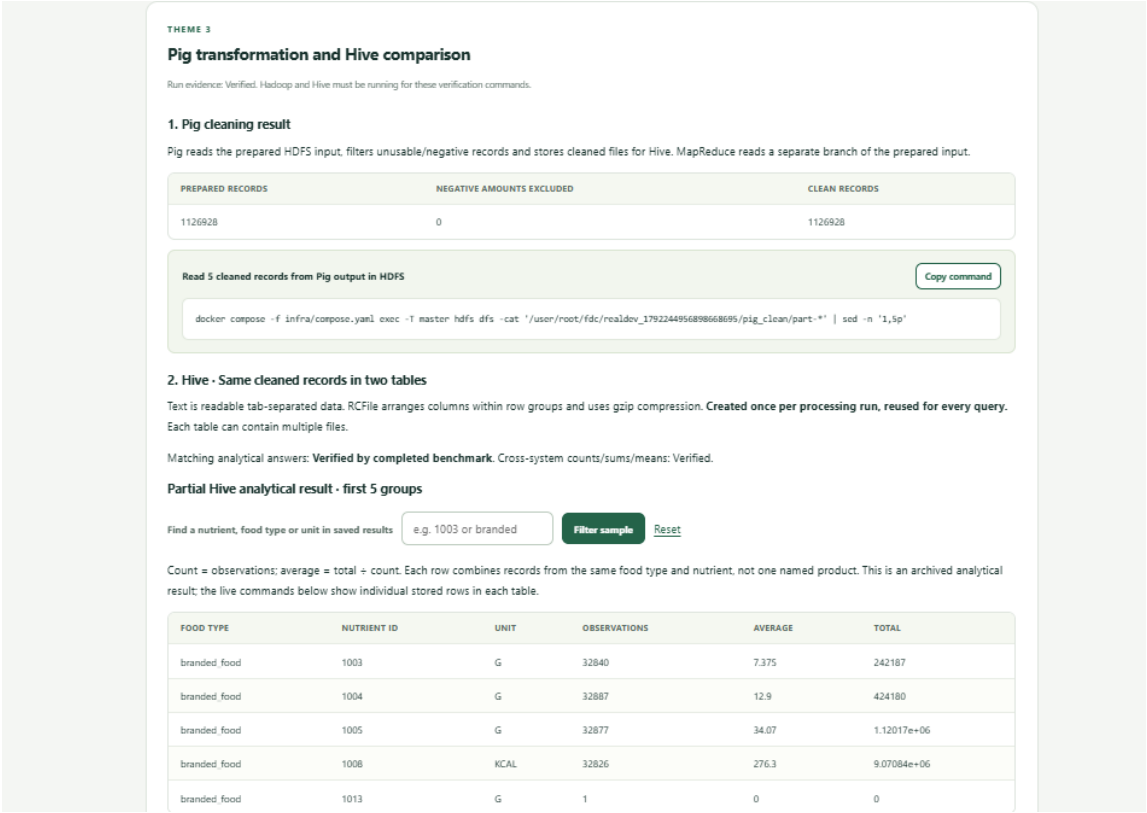


Figure 5. Pig cleaning and Hive results for the same 1,126,928 observations.

Text and RCFile contain the same cleaned rows and are reused across query repetitions. The dashboard shows count, total and mean for each food-type/nutrient/unit group. Users can filter the visible sample and copy read commands for HDFS, Pig output and Hive tables. These buttons copy commands rather than executing them automatically.

6. Performance Evaluation



4. Index performance

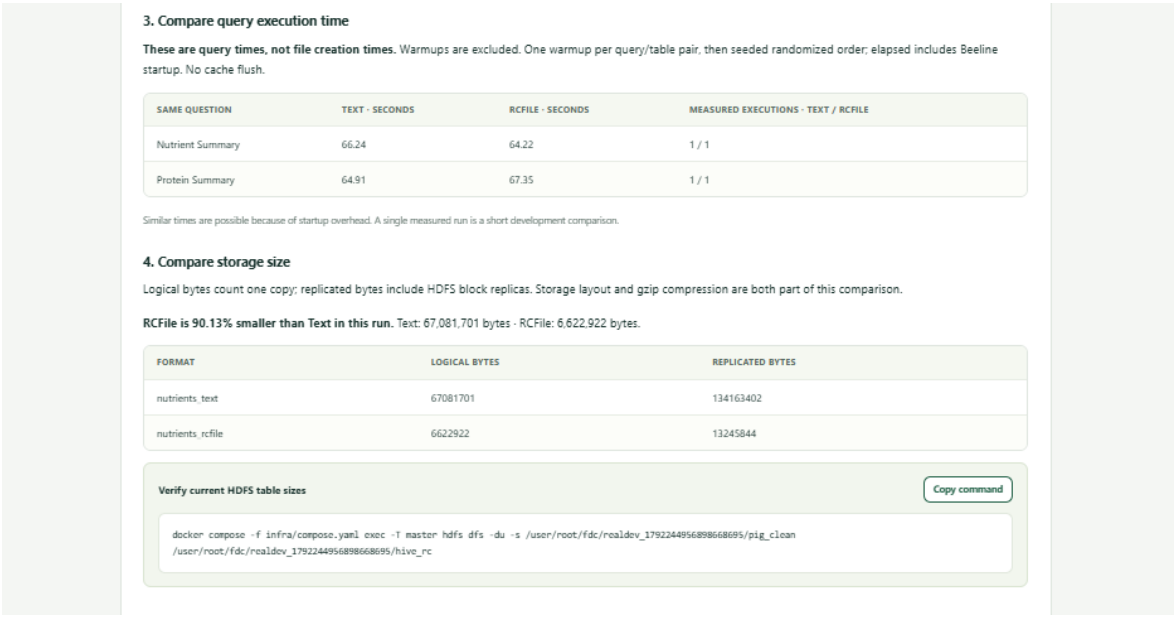
The same food lookup is repeated five times per plan. Documents examined means records checked; Returned means matching records found. A 0 ms value is below the timer's millisecond resolution.

PLAN	RUNS	DOCUMENTS EXAMINED	INDEX KEYS	RETURNED	MEDIAN SERVER MS
collection_scan	5	50000	0	1	42
indexed	5	1	1	1	0

Verify the indexed lookup with executionStats Copy command

```
docker exec fdc-mongo-dev mongosh --quiet --eval 'const d=db.getSiblingDB("fdc_ui50000_2d4ef797bae5466d"); const f=d.raw_food.findOne(); if(f)
printjson(d.runCommand({explain:{find:"raw_food",filter:{fdc_id:f.fdc_id},hint:"idx_fdc_id"},verbosity:"executionStats"}))'
```

Figure 6. MongoDB index evidence: five measurements per plan.



3. Compare query execution time

These are query times, not file creation times. Warmups are excluded. One warmup per query/table pair, then seeded randomized order; elapsed includes Beeline startup. No cache flush.

SAME QUESTION	TEXT · SECONDS	RCFILE · SECONDS	MEASURED EXECUTIONS · TEXT / RCFILE
Nutrient Summary	66.24	64.22	1 / 1
Protein Summary	64.91	67.35	1 / 1

Similar times are possible because of startup overhead. A single measured run is a short development comparison.

4. Compare storage size

Logical bytes count one copy; replicated bytes include HDFS block replicas. Storage layout and gzip compression are both part of this comparison.

RCFile is 90.13% smaller than Text in this run. Text: 67,081,701 bytes · RCFile: 6,622,922 bytes.

FORMAT	LOGICAL BYTES	REPLICATED BYTES
nutrients_text	67081701	134163402
nutrients_rcfile	6622922	13245844

Verify current HDFS table sizes Copy command

```
docker compose -f infra/compose.yaml exec -T master hdfs dfs -du -s /user/root/fdc/realdev_17922449568986868695/pig_clean
/user/root/fdc/realdev_17922449568986868695/hive_rc
```

Figure 7. Hive timing and storage evidence for the selected development run.

MongoDB examines one document with the index instead of 50,000, with median server times of 0 ms and 42 ms. Zero is below timer resolution, not zero work. Hive's all-nutrient times are 66.24 / 64.22 s and protein times are 64.91 / 67.35 s (Text / RCFile). Warmups are excluded; each pair has one measured execution.

RCFile reduces logical storage from 67,081,701 to 6,622,922 bytes, approximately 90.13%. The time comparison varies by query. This interface therefore presents recorded measurements without treating storage savings as proof of a universal speedup.

7. Conclusion and References

The Flask extension brings the USDA pipeline into one demonstrable interface. It lets users launch permitted laptop workflows, monitor their progress, select completed runs and explain the three assignment themes through actual data samples and measured outputs.

The supplied screenshots document the linked 50,000-food development run. Before-and-after views show the transformation from source records to answers, while adjacent commands support direct source verification. The application preserves the distinction between historical results, current service state and benchmark measurements, making both the processing and the evidence easier to present.

References

1. USDA FoodData Central — Download Datasets. https://fdc.nal.usda.gov/download-datasets/

2. Flask — Documentation. https://flask.palletsprojects.com/

3. MongoDB — Aggregation Pipelines. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

4. Apache Hadoop — Cluster Setup. https://apache.github.io/hadoop/hadoop-project-dist/hadoop-common/ClusterSetup.html

5. Apache Hive — Language Manual. https://cwiki.apache.org/confluence/display/Hive/LanguageManual

6. Project source code. https://github.com/Dhanush-singh-g/BDA-Assignment